ETL Data Pipelines in Modernen Datenarchitekturen

Ingestion • Transformation • Orchestrierung • Betrieb • Skalierung

Warum ETL heute geschäftskritisch ist

- Entscheidungen hängen an verlässlichen Daten
- BI, ML/AI und operative Systeme greifen auf dieselben Pipelines zu
- Compliance, Security und Auditierbarkeit steigen in Priorität

Business-Impact von guten Pipelines

- schnellere Insights
- weniger Incident-Kosten
- höhere Vertrauenstiefe bei Stakeholdern
- kürzere Time-to-Market für Datenprodukte

ETL

- **Extract**: Daten erfassen
- **Transform**: Daten nutzbar machen
- **Load**: Daten zielgerichtet bereitstellen

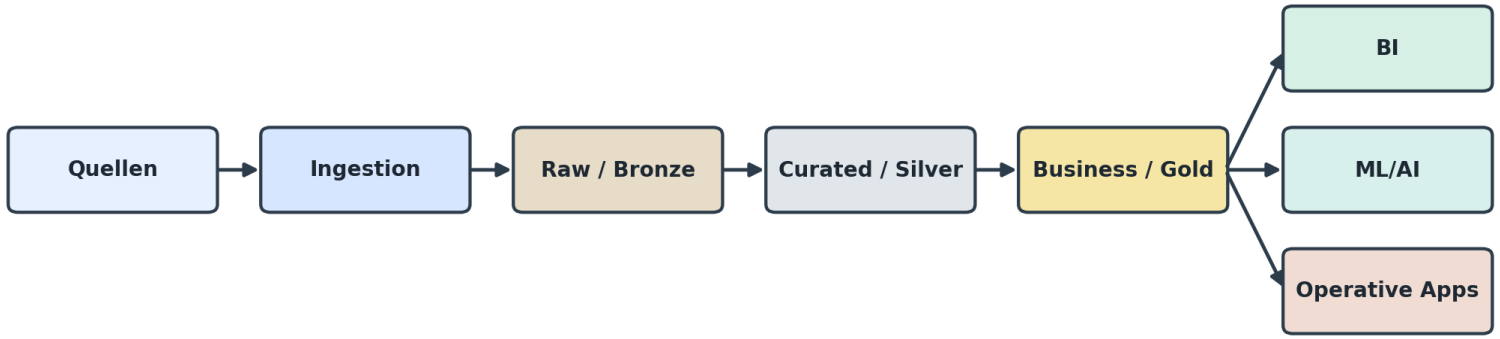
ETL ist kein Job – es ist ein System.

ETL vs ELT vs Reverse ETL

- **ETL**: Transform vor dem Laden
- **ELT**: Laden vor der Transformationslogik
- **Reverse ETL**: kuratierte Daten zurück in operative Tools

Praxis: häufig hybride Kombination.

Grundmuster



Leitprinzipien für alle Entscheidungen

- reliability first
- idempotent by default
- observability everywhere
- standardize what repeats
- optimize with metrics, not opinions

Data Ingestion

Ziel:

- viele Quellen konsistent onboarden
- schema- und versionsrobust bleiben
- klare Service Level Objectives (SLOs) für Frische und Vollständigkeit sichern

Source-Assessment: Pflichtfragen

- Wer sind Owner und Stakeholder?
- Wofür wird die Quelle fachlich genutzt?
- Welche Frequenz ist real erforderlich?
- Bounded oder unbounded Datenfluss?

Source-Assessment: Pflichtfragen

- Volumen jetzt vs. in 12 Monaten?
- Format und Strukturgrad (JSON/CSV/DB/Event)?
- Qualität und bekannte Schwachstellen?
- Sicherheits-/PII-Einstufung (PII = Personally Identifiable Information)?

Ingestion-Muster

- batch (planbar, kosteneffizient)
- micro-batch
- streaming (niedrige Latenz, höherer Betriebsaufwand)
- CDC = Change Data Capture (effizient bei Änderungsdaten)

Batch vs Streaming: Entscheidungsraster

Batch wenn:

- Stunden-/Tageslatenz genügt
- Kosten und Einfachheit priorisiert sind

Streaming wenn:

- Minuten-/Sekundenreaktion nötig ist
- kontinuierliche Ereignisverarbeitung Business-kritisch ist

Ingestion-Frequenz als Produktentscheidung

- nicht „was technisch geht“
- sondern „was fachlich wirklich gebraucht wird“

Faustregel: so selten wie möglich, so häufig wie nötig.

Schema-Drift beherrschen

- Versionierung von Contracts
- Breaking Changes explizit kommunizieren
- Validierung bei Ingestion (Schema + Feldregeln)
- Quarantäne für ungültige Datensätze

Ingestion-Sicherheit

- least privilege für Jobs
- Secret Management statt Hardcoded Credentials
- Verschlüsselung in transit + at rest
- Audit-Logs für Zugriffe und Pipeline-Runs

Ingestion Anti-Patterns

- jede Quelle individuell „special cased”
- fehlende Ownership pro Source
- keine Backfill-Strategie
- Alerts erst beim Fachbereich sichtbar

Ingestion Done-Definition

- source checklist vollständig
- klare SLOs (Freshness/Completeness)
- Monitoring + Alerts aktiv
- dokumentierter Onboarding-Runbook

Data Transformation

Ziel:

- aus Rohdaten verlässliche Datenprodukte machen
- Fachlogik reproduzierbar und testbar betreiben
- Änderungen sicher ausrollen

Transformation als mehrstufiger Prozess

- Rohdaten normalisieren
- Daten bereinigen und anreichern
- Fachlogik modellieren
- aggregieren und bereitstellen

Transformationsumgebung wählen

- Warehouse-zentriert (SQL-first)
- Lake + externes Compute
- Lakehouse (einheitlicherer Operating Layer)

Entscheidung nach Team, Datenprofil, Kostenmodell.

SQL vs Python

- SQL: stark für tabellarische Logik und Governance
- Python: stark für komplexe/algorithmische Verarbeitung
- starke Plattformen kombinieren beides sauber

Update-Strategien

- full refresh
- incremental insert
- upsert/merge
- CDC-basiertes Aktualisieren

Kein Pattern ist „immer richtig“.

Idempotenz

Jeder Lauf muss wiederholbar sein ohne Seiteneffekte:

- stabile Keys
- deterministische Business-Logik
- kontrollierte Merge-Policies

Ein erneuter Lauf mit denselben Inputs erzeugt denselben Zielzustand, ohne Duplikate oder widersprüchliche Updates!

Data Quality Gates in Transformation

- uniqueness checks
- null/mandatory checks
- domain checks (Wertebereiche)
- referential checks

Fehler früh stoppen!

Tests

- unit tests für Kernlogik
- data tests für Modelle
- regression checks vor Deployment
- smoke tests direkt nach Run

Transformation Anti-Patterns

- monolithische „alles in einem Job“-Pipelines
- ungeklärte Feldsemantik
- Reports auf Roh-/Silver-Tabellen
- keine Ownership pro Gold-Modell

Transformation Done-Definition

- klare Layer-Zuordnung
- dokumentierte Business-Definitionen
- automatisierte Tests/Gates
- reproduzierbarer Backfill möglich

Data Orchestrierung

Ziel:

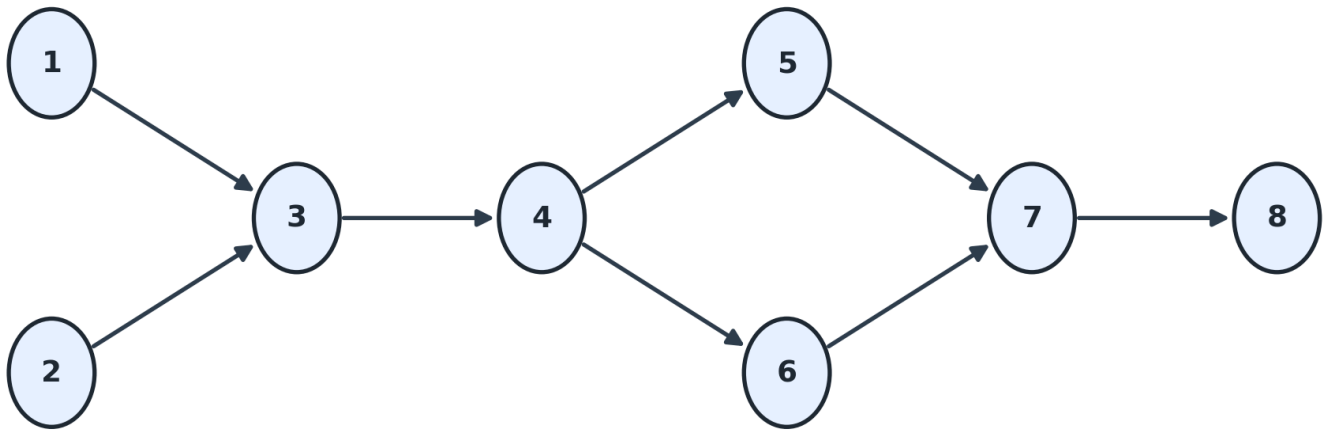
- abhängige Tasks robust und sichtbar steuern
- Fehlerbehandlung und Recovery standardisieren
- Laufzustände nachvollziehbar halten

Orchestrator vs Scheduler

- Scheduler: Zeitsteuerung
- Orchestrator: Zeit + Events + Abhängigkeiten + Recovery + Monitoring

DAG-Grundlagen

- Node = Task
- Edge = Dependency
- gerichteter, zyklusfreier Ausführungsgraph


1
2
3
4
5
6
7
8

Trigger-Modelle

- zeitbasiert (cron)
- event-driven (Webhook/Queue/File arrival)
- dependency-driven (upstream success)

Orchestrierungsmuster

- task decomposition
- retries mit backoff (wachsende Wartezeit)
- dead letter handling (separate Fehlerablage)
- idempotent reruns
- run metadata + lineage links

Rolle des Orchestrators

Best practice:

- Orchestrator steuert
- spezialisierte Services rechnen
- klare Trennung von Workflowlogik und Verarbeitungscode

Auswahlkriterien

- Skalierbarkeit
- Integrationen
- Betriebsmodell (managed/self-hosted)
- Team-Fit & Lernkurve
- Debuggability & Observability

Orchestrierung Done-Definition

- DAGs klein, modular, nachvollziehbar
- Retry-/Timeout-Konventionen etabliert
- Alerts mit Owner-Routing
- Runbook pro kritischer Pipeline

Pipeline-Stabilität & Troubleshooting

Ziel:

- Vorfälle schneller erkennen und eingrenzen
- mittlere Wiederherstellungszeit reduzieren
- systematisch aus Fehlern lernen

Wartbarkeit als Wirtschaftsfaktor

- direkte Kosten: Compute, Laufzeit, Ops-Aufwand
- indirekte Kosten: Ausfallfolgen, Vertrauensverlust, Team-Burnout
- Wartbarkeit ist keine „Nice-to-have“-Eigenschaft.

Observability-Stack

- logs
- metrics
- traces
- lineage
- alerting

Quality-Metriken konkret

- uniqueness
- completeness
- validity
- consistency
- referential integrity

Lineage für Incident-Response

- schneller Root Cause
- Impact-Analyse (welche Reports betroffen?)
- kontrollierte Kommunikation an Stakeholder

Präventive Kontrollen

- anomaly detection
- data diff checks
- assertions
- schema checks bei Ingestion

Error Handling Patterns

- conditional branching
- retries mit sinnvollen Grenzen
- circuit breaker bei externen Quellen
- partial failure isolation

Graceful Degradation

Wenn ein Teil ausfällt:

- Restsystem bleibt funktionsfähig
- betroffene Produkte klar kommuniziert
- Recovery priorisiert nach Business-Impact

Incident-Lifecycle

1. detect
2. triage
3. mitigate
4. recover
5. postmortem
6. preventive action

Postmortem-Template

- was ist passiert?
- warum ist es passiert?
- warum wurde es nicht früher erkannt?
- welche Guardrails fehlen?
- welche Maßnahmen bis wann, mit Owner?

Effizienz, Skalierung, Kosten

Ziel:

- mehr Last verarbeiten
- stabile Qualität halten
- Kosten kontrolliert entwickeln

Compute-Strategien

- on-demand (stabil, teurer)
- spot/preemptible (günstiger, unterbrechbar)
- pool/shared clusters
- serverless (weniger Ops, andere Kostenkurve)

Rollen

- Platform/Data Infra
- Data Engineering
- Analytics Engineering
- Data Governance/Stewardship
- Product/Business Owner